LYAPUNOV system design

Distributed event reliability and fault-verification platform. Java 21, Maven, PostgreSQL 16, Apache Kafka. Design review draft, 11 October 2026.

Executive summary

LYAPUNOV is an engineering tool for checking whether an event-driven system actually delivers its events correctly. It does not stop at "the producer returned success". It asks whether every expected event was observed, observed once, and observed in order, both in normal operation and under deliberately injected failures.

Events are persisted in a PostgreSQL outbox, published to Kafka, observed by a separate consumer and compared with the expected set. A seeded fault injector (DROP, DUPLICATE, REORDER) proves the verifier can detect the failures it claims to detect. Every experiment writes a JSON report. Reports can be replayed from their seed, compared against a baseline, and enforced as a regression gate in CI.

Verified today

Unit tests: 21 run, 0 failures, 0 errors, 0 skipped, covering RegressionGateTest, ExperimentComparatorTest, EventVerifierTest, FaultInjectorTest and FaultInjectionExperimentTest.
Regression gate manually exercised: missing delta -1, duplicate delta 0, ordering delta +1, outcome MIXED, gate FAIL.
Current report writer output parsed as valid JSON.

Not claimed

Exactly-once delivery or processing.
Strict per-aggregate ordering. claimPendingEvents does not enforce head-of-line ordering today, so this is a known implementation gap.
End-to-end PostgreSQL to Kafka validation. The Docker engine was unavailable, so Docker-dependent integration tests are unverified.

Problem statement

In a distributed system a successful produce or consume call does not prove correct end-to-end processing. Events can be lost or never observed, delivered more than once, observed in an unexpected order, processed inconsistently across components, or disturbed by failures during publication or processing. LYAPUNOV treats five stages as five separate pieces of evidence. They are never treated as equivalent.



Only stage 5 produces a verdict. Stages 1 to 4 are inputs to it.

How to read this canvas

Unit-tested | Exists, not recently validated | Known gap | Proposed | External

Status colours mark what is true today and are never decoration. Solid teal means covered by the 21 passing unit tests. Blue means the component exists but its Docker-dependent paths have not been validated recently. Red marks a known gap. Dashed amber is proposed work, not implemented. Grey is external infrastructure. Boards 03 to 06 give architecture and data, 07 to 11 give behaviour, 12 to 15 give reliability, failures, testing and operations, and 16 to 18 give status, roadmap and risks.

Requirements

Functional requirements describe what LYAPUNOV does. Non-functional requirements describe how well. Status reflects the current repository as described in the project brief.

Functional requirements

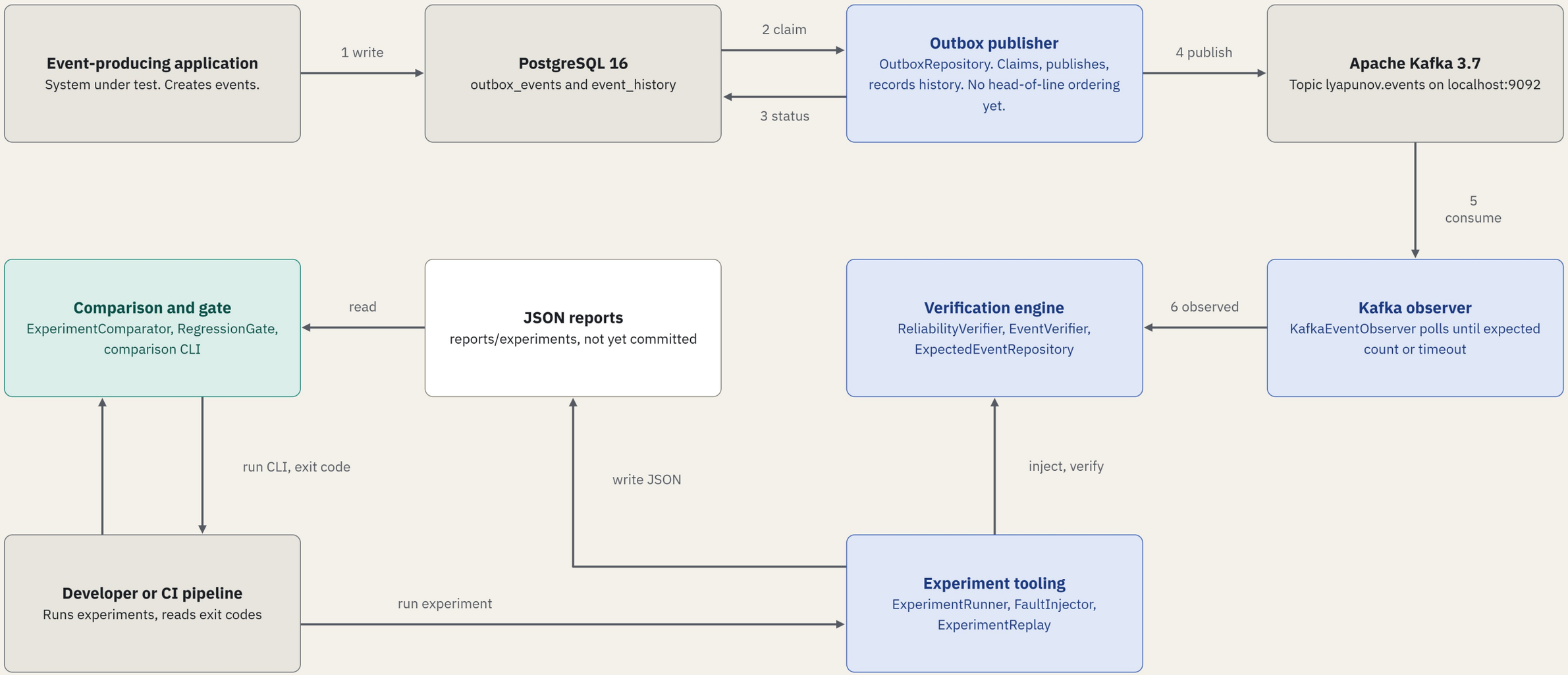
ID	REQUIREMENT	STATUS
FR1	Create events with a stable ID, type, aggregate or entity ID, sequence where applicable, payload and creation timestamp.	Exists
FR2	Persist events in PostgreSQL outbox_events and record each lifecycle transition in event_history.	Exists
FR3	Claim PENDING events and publish them to Kafka topic lyapunov.events, moving them to PUBLISHED or FAILED.	Exists
FR4	Observe events consumed from Kafka until the expected count is reached or a timeout expires.	Exists
FR5	Hold the expected event set separately from observations (ExpectedEventRepository).	Exists
FR6	Detect missing event IDs, duplicate event IDs and ordering violations.	Tested
FR7	Inject DROP, DUPLICATE and REORDER faults deterministically from a seed.	Tested
FR8	Run an experiment from seed, fault type, event count and optional report directory, then write a JSON report.	Exists
FR9	Replay an experiment from the seed, fault type and count saved in a report.	Exists
FR10	Compare baseline and candidate reports with delta = candidate minus baseline, classified IMPROVED, REGRESSED, MIXED or UNCHANGED.	Tested
FR11	Fail the build when any delta exceeds its threshold. --fail-on-regression applies zero-increase thresholds.	Tested
FR12	Summarise a directory of reports, isolate malformed ones, and support a strict mode that exits non-zero on any malformed report.	Proposed
FR13	Enforce per-aggregate head-of-line ordering when claiming outbox events.	Gap

Non-functional requirements

ID	REQUIREMENT	STATUS
NFR1	Reproducibility. The same seed, fault type and count must reproduce the same injected faults. Timing and infrastructure effects are reported, not hidden.	Exists
NFR2	Honesty of claims. Never state exactly-once delivery or strict ordering before they are enforced and tested.	Exists
NFR3	Validation over silence. Malformed or incomplete reports are failures, never counted as zero.	Proposed
NFR4	Maintainability. Keep the single Java and Maven codebase. No new framework or infrastructure without a written justification.	Exists
NFR5	Testability. Pure logic (verifier, injector, comparator, gate) is testable without Docker.	Tested
NFR6	Configurability. Kafka and JDBC settings, credentials and thresholds come from external configuration, not source.	Gap
NFR7	Observability. Structured logs, health checks and metrics for long-running components.	Proposed
NFR8	CI integration. Exit codes are reliable so pipelines can gate on them.	Exists

System context

Who and what LYAPUNOV talks to. Arrows show the direction of data. Grey cards are external to LYAPUNOV; coloured cards are LYAPUNOV components.

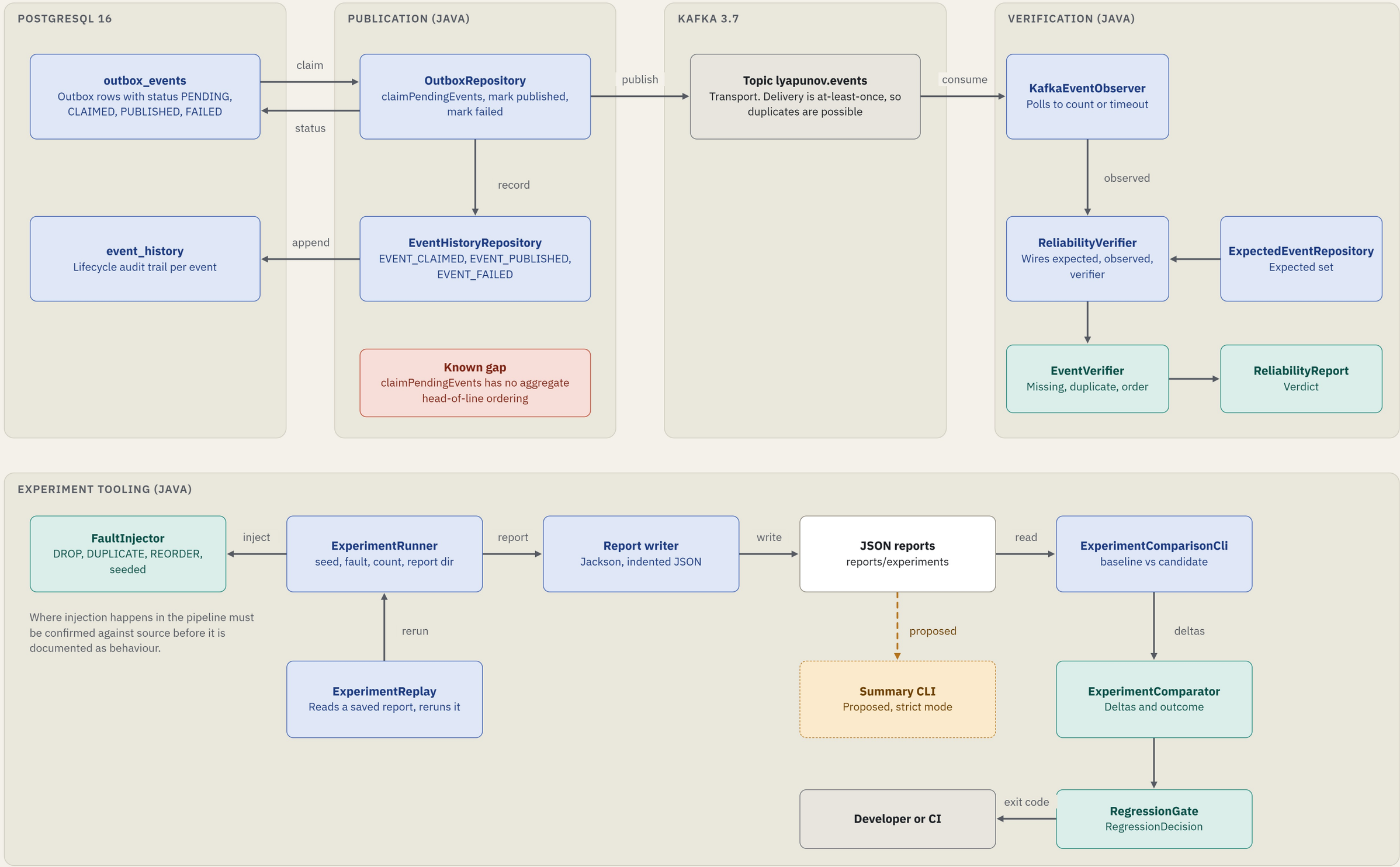


Legend

Unit-tested | Exists, not recently validated | External | Artefact or data

Component architecture

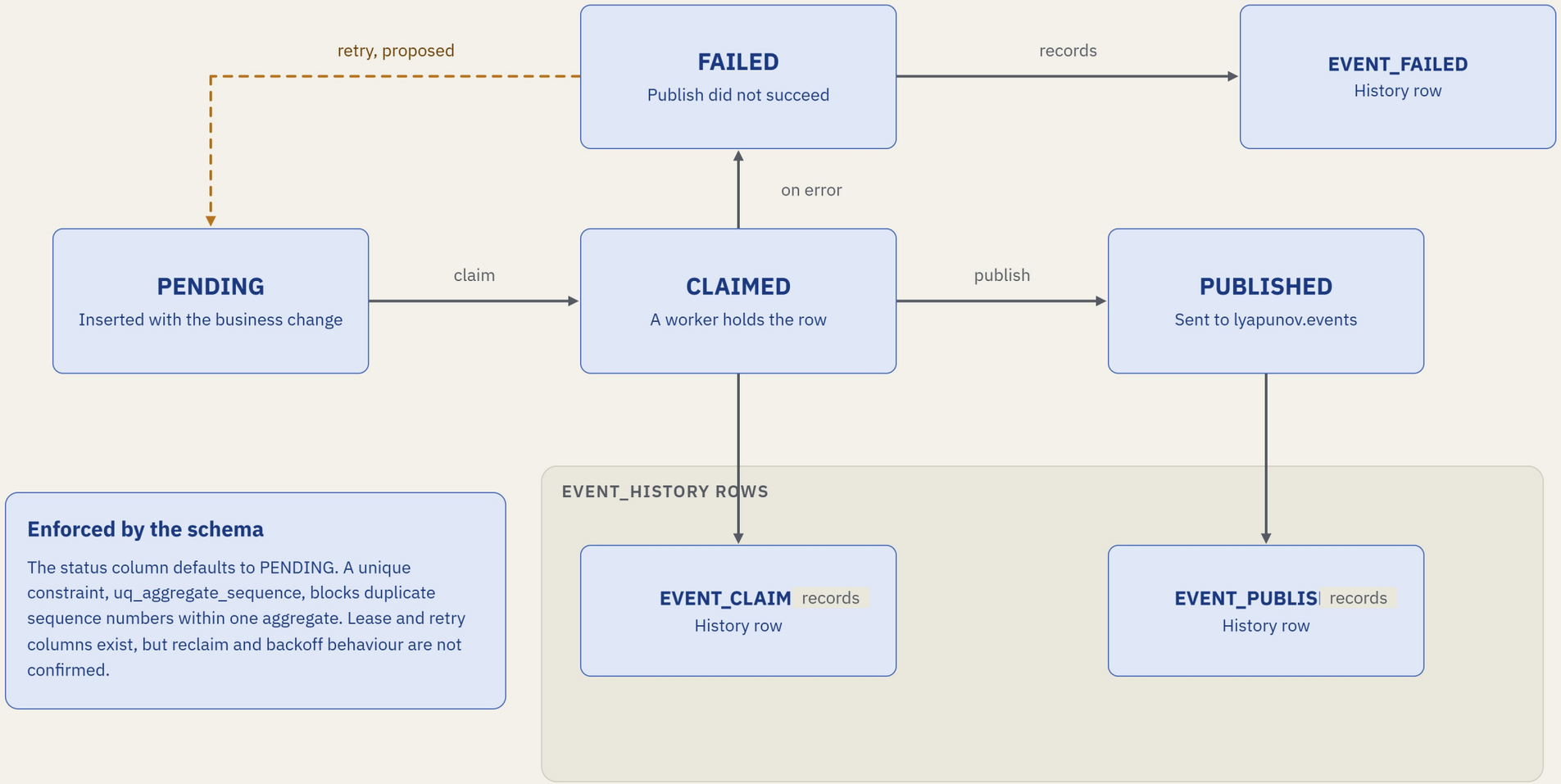
Logical components grouped by responsibility. Four runtime areas on top, experiment tooling below. Arrows show the direction of data or control.



Unit-tested | Exists, not recently validated | Known gap | Proposed | External

Event lifecycle

Outbox status transitions and the history rows they write. Solid arrows are the intended lifecycle. The dashed amber arrow is proposed behaviour.



Enforced by the schema

The status column defaults to PENDING. A unique constraint, uq_aggregate_sequence, blocks duplicate sequence numbers within one aggregate. Lease and retry columns exist, but reclaim and backoff behaviour are not confirmed.

Not enforced today

Head-of-line ordering per aggregate. Exactly-once delivery. If a worker crashes after Kafka accepts a message but before the row is marked PUBLISHED, the event can be published twice. That is at-least-once, and the verifier measures it.

Exists, not recently validated | Known gap | Proposed

Database design

Source of truth is docker/init.sql. Fields below are summarised from it. Confirm exact column names and types in the file before writing code against them.

outbox_events

Exists, not recently validated

FIELD	PURPOSE
event ID	Stable identifier. The verifier matches on this.
aggregate type, aggregate ID	Which entity the event belongs to.
sequence number	Position within the aggregate. Basis for ordering checks.
event type	Kind of event.
payload (JSONB)	Event body.
status	PENDING by default, then CLAIMED, PUBLISHED or FAILED.
retry counter	Attempts so far.
claiming worker, lease expiry	Which worker holds the row and until when.
created at, updated at	Timestamps.

event ID

event_history

Exists, not recently validated

FIELD	PURPOSE
event ID	The event this row describes.
aggregate type, aggregate ID, sequence number	Copied for audit queries by aggregate.
event type	Kind of event.
action	EVENT_CLAIMED, EVENT_PUBLISHED, EVENT_FAILED.
worker ID (optional)	Which worker acted.
details (JSONB, optional)	Free-form context such as an error.
occurred at	When the action happened.

Constraints and indexes

outbox_events

Unique constraint uq_aggregate_sequence: one sequence number per aggregate.
Partial index on created at for PENDING rows, supporting the claim poll.
Composite index on aggregate type and ID.

event_history

Composite index on event ID and occurred at.
Composite index on aggregate type, aggregate ID, sequence number and occurred at.

Not confirmed from the schema

Whether event_history.event ID is a declared foreign key. Whether expected events for the verifier live in PostgreSQL at all. Check ExpectedEventRepository before documenting it.

Proposed additions (not implemented)

dead_letter_events

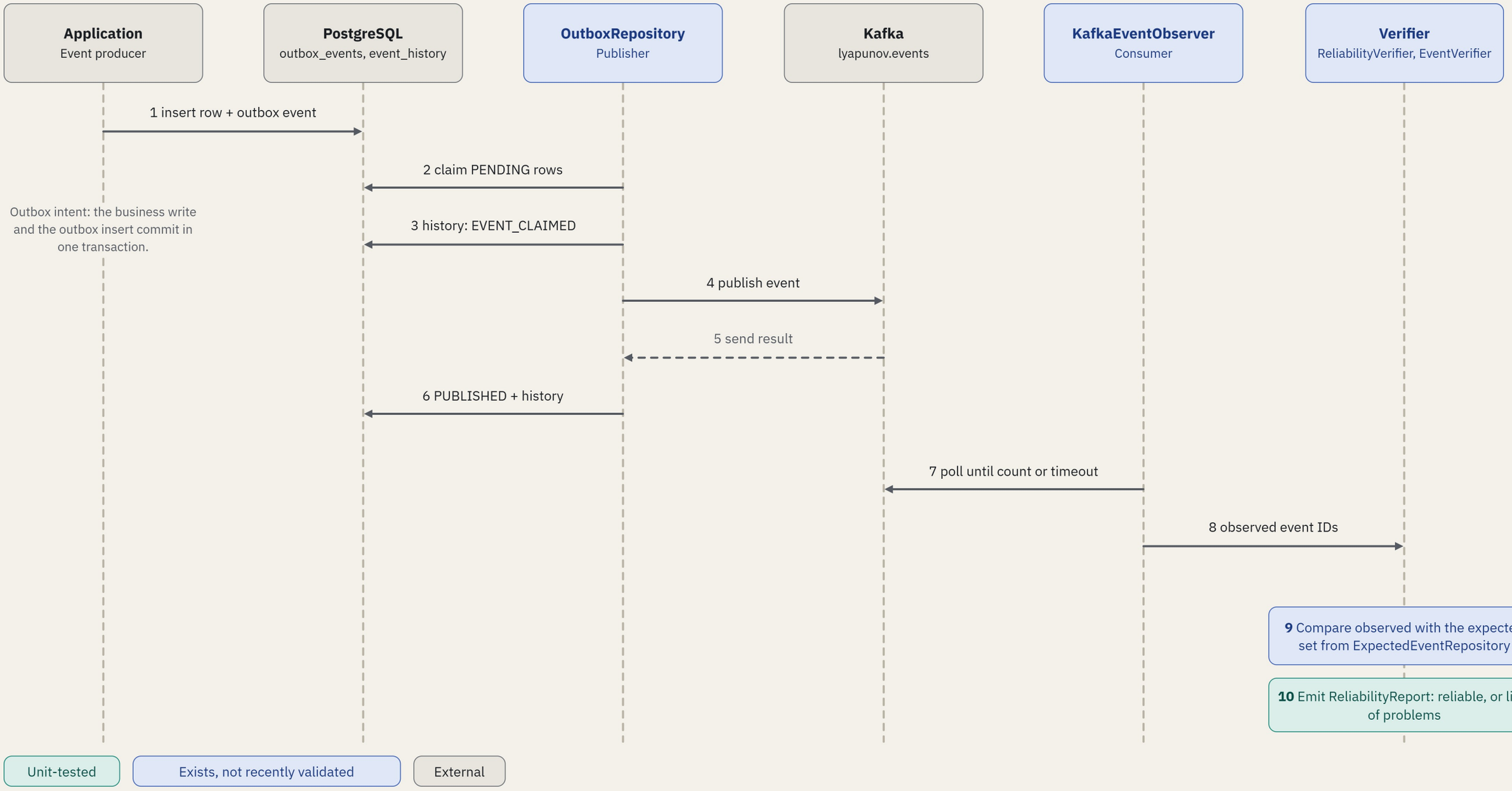
Holds events that exhausted their retries so they stop blocking a worker and can be inspected or replayed. Needed for Stage 4 of the roadmap. Alternative: a DEAD status on outbox_events plus a history action, with no new table. Prefer that first, since it adds no schema.

experiment_runs

Index of experiment reports (seed, fault type, counts, verdict, file path) so history is queryable. Only worth adding if report directories become unmanageable. Today JSON files are enough and need no database.

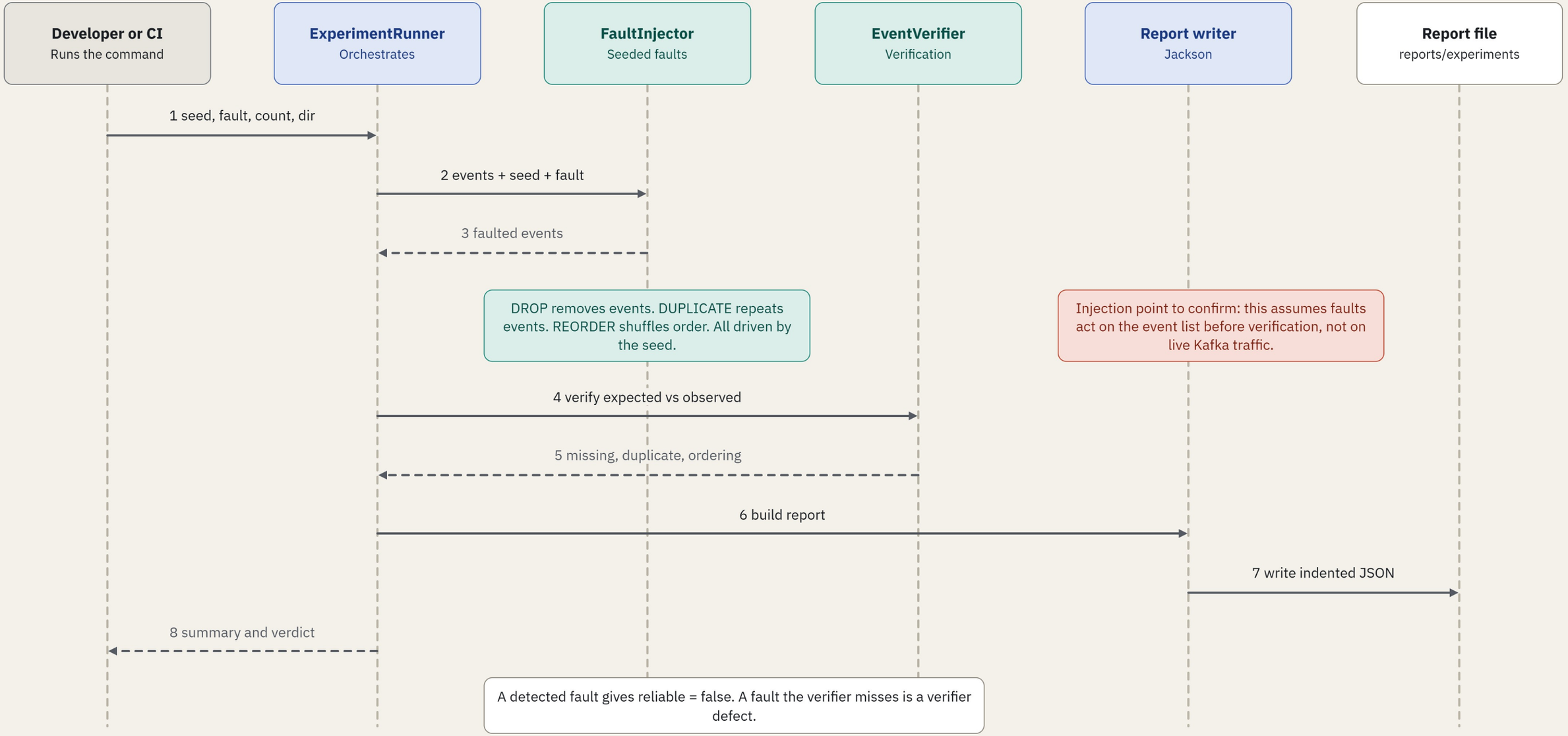
Sequence 1: normal publication and verification

The happy path from a persisted event to a ReliabilityReport. Steps 2 to 7 touch PostgreSQL and Kafka, so they have not been validated recently (Docker engine unavailable).



Sequence 2: fault-injection experiment

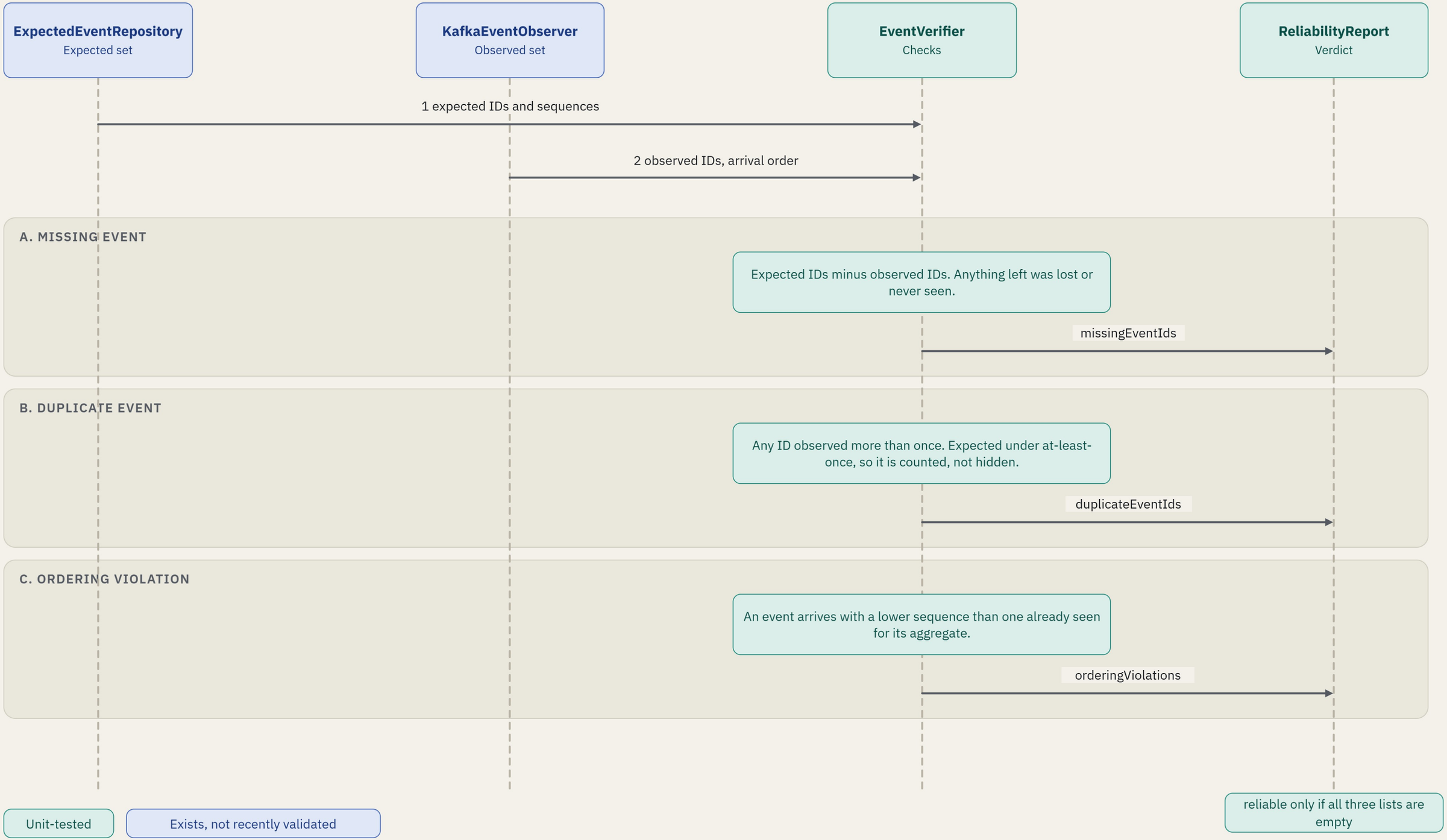
Run: seed, DROP, DUPLICATE or REORDER, and an event count. The injector is deterministic for a given seed, so the same run can be repeated.



Unit-tested | Exists, not recently validated | Known gap | Artefact or data

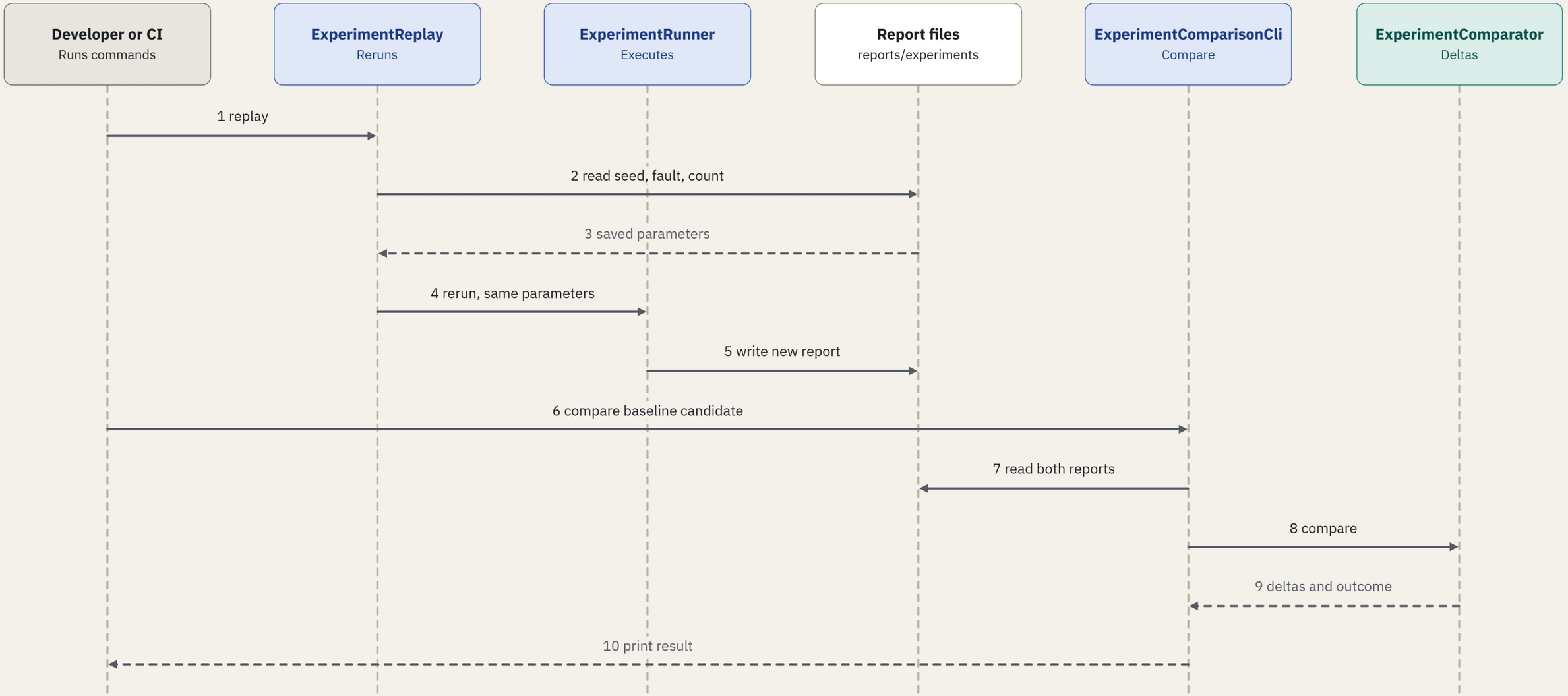
Sequence 3: missing, duplicate and ordering detection

One verification pass, shown as the three checks EventVerifier performs. Each check fills its own list in ReliabilityReport. The rules below are the design intent. Confirm exact comparisons in EventVerifier.



Sequence 4: experiment replay and baseline comparison

Replay reruns a saved experiment from its seed. Comparison reads two reports and computes deltas as candidate minus baseline.



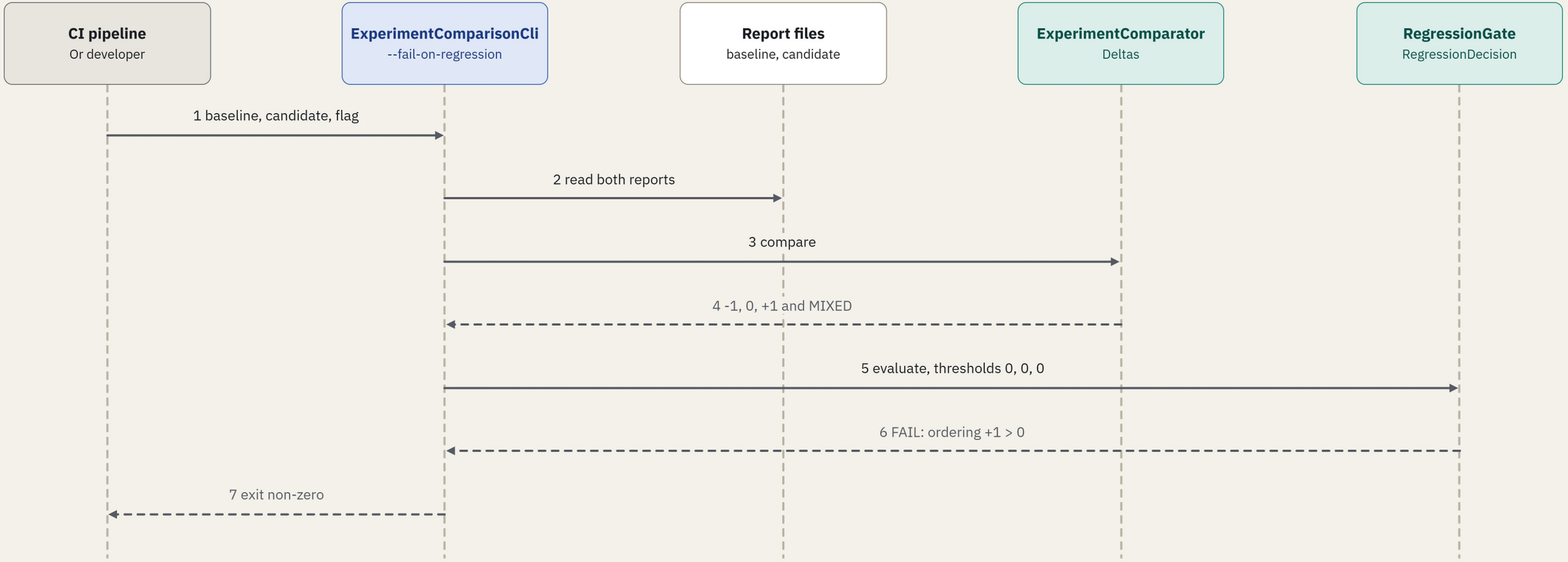
Deterministic: seed, which events are dropped or duplicated, the shuffle order. **Can still vary:** timestamps, Kafka and database state, timing and timeouts. Replay reproduces the injected fault, not the environment.

delta = candidate minus baseline. Negative improves, positive regresses, zero is unchanged. MIXED means one metric improved while another regressed.

Unit-tested | Exists, not recently validated | Artefact or data | External

Sequence 5: regression-gate failure

A candidate that improves one metric but regresses another is still rejected. Values are from the manual run recorded in the project brief.



METRIC	DELTA	ALLOWED INCREASE	RESULT
Missing events	-1	0	Pass, improved
Duplicate events	0	0	Pass, unchanged
ns			ressed

Unit-tested | Exists, not recently validated | Artefact or data | External

Why MIXED still fails

Strict thresholds allow no increase on any metric. A gain in one place never pays for a loss in another. Configurable thresholds already exist in RegressionGate. The CLI flag applies the strict zero-increase set.

Reliability design

What LYAPUNOV means by each reliability term, and how each one is checked. Status follows the project brief, not an inspection of the source.

Delivery semantics

TERM	MEANING	LYAPUNOV TODAY	STATUS
At-least-once delivery	Every committed event is published one or more times. Duplicates are possible after a crash or retry.	Target model. Not proven end to end yet.	Exists
Duplicate delivery	The same event ID reaches a consumer more than once.	Detected and counted by EventVerifier.	Tested
Idempotent processing	A consumer applies a duplicate without changing the result, usually by remembering event IDs.	Not part of LYAPUNOV. Consumers need it.	Proposed
Exactly-once processing	At-least-once delivery plus idempotent processing, or transactions end to end.	Not claimed.	Gap
Database and Kafka consistency	The two cannot commit atomically. The outbox makes the database the source of truth and publishes from it.	Outbox in place. Crash windows unvalidated.	Exists

Verification design

CONCEPT	DESIGN	OPEN POINT	STATUS
Event identity	Every event carries a stable event ID, created once and never reused. Matching on ID, not on position or payload, is what makes loss and duplication measurable. Aggregate ID and sequence number form the ordering key.	Final schema to be checked against the code.	Exists
Expected vs observed	Expected is what LYAPUNOV intends to see. Published is what the outbox believes it sent. Observed is what the consumer read. Verified is the verdict. They stay four separate sets.	Where expected events are stored is unconfirmed.	Exists
Missing-event detection	Expected IDs minus observed IDs. Each leftover ID is reported by name.	A poll timeout can make a slow event look missing. Record timeout as its own cause.	Tested
Duplicate detection	Any ID seen more than once is reported with its count.	None.	Tested
Ordering verification	Per aggregate, sequence numbers must not go backwards in arrival order. Kafka orders only within a partition, so the partition key should be the aggregate ID.	Claim order and partition key are not enforced today.	Gap
Deterministic fault injection	The seed fully determines which events are dropped, duplicated or moved. Same seed, fault and count give the same faulted stream.	Confirm where injection sits in the pipeline.	Tested
Reproducible execution	A report stores seed, fault type, count and time. Replay reads them back and reruns. Timing and infrastructure remain outside the seed.	Report executedAt is a String, not a typed timestamp.	Exists
JSON report validation	Three levels. Valid JSON. Structurally valid report, with every required field present and correctly typed. Then a reliable or unreliable verdict. Missing fields are errors, never zero.	Four older reports are truncated and must be flagged, not deleted or counted.	Proposed
Baseline and candidate comparison	delta = candidate minus baseline for missing, duplicate and ordering counts. Outcome is IMPROVED, REGRESSED, MIXED or UNCHANGED.	None.	Tested
Regression thresholds	Per-metric allowed increase. Strict mode is zero on all three. Future tolerances for performance metrics, never for missing events.	Threshold source is CLI-only today.	Tested

Failure scenarios and recovery strategy

How LYAPUNOV should reason about each failure. The Today column is existing behaviour as described in the brief. The Recommended column is proposed work, not implemented.

SCENARIO	HOW LYAPUNOV REASONS ABOUT IT	TODAY	RECOMMENDED, PROPOSED
Database failure	A claim or status update throws. The row stays PENDING or CLAIMED, so nothing is lost: the row is the source of truth.	Errors surface as exceptions. Recovery behaviour is not confirmed.	Bounded retry with backoff. Pool health check. Reclaim CLAIMED rows once their lease expires.
Kafka unavailable	Publish fails. The row must not be marked PUBLISHED. Record EVENT_FAILED.	EVENT_FAILED history and FAILED status exist. No retry policy is confirmed.	Exponential backoff, a cap on attempts, then dead-letter. Explicit producer timeouts.
Publication failure	A timeout is ambiguous. The broker may have received the message. Treat it as maybe published.	FAILED status.	Retry is safe only because consumers must tolerate duplicates. Say so in the docs.
Consumer interruption	The observer stops before the expected count. That is a timeout, not proof of loss.	Observer polls until count or timeout.	Report "not yet observed" separately from "missing". Resume from committed offsets.
Missing events	Expected IDs not observed. Listed by name.	Detected by EventVerifier. Unit-tested.	Break down by cause: dropped by injected fault, lost in transport, timed out.
Duplicate events	Expected under at-least-once. A failure only if the consumer effect is not idempotent.	Detected and counted. Unit-tested.	Add an idempotent sample consumer so the effect, not only the count, is verified.
Event reordering	Sequence goes backwards within an aggregate.	Detected. Claim order is not enforced, so the cause is not prevented.	Head-of-line claim per aggregate. Partition key is the aggregate ID. One in-flight event per aggregate.
Invalid experiment reports	Malformed JSON or missing required fields.	Four truncated reports exist locally. Nothing validates them yet. Do not delete them.	Summary CLI validates each file, names bad ones, continues with the rest, and exits non-zero in strict mode. Never silently zero.
Regression-gate failure	Any metric above its allowed increase fails the build.	Exits unsuccessfully with --fail-on-regression.	Distinct exit codes for "gate failed" and "tool error". Publish the comparison as a CI artifact. Require a baseline.
Timeout while waiting	Fewer events than expected before the deadline.	The poll ends at the timeout.	Configurable timeout. Record elapsed time and last offset. One retry before the verdict.

Testing architecture

Unit tests cover pure logic and run without Docker. Integration tests need PostgreSQL and Kafka and are unverified until the Docker engine works again.

What the 21 passing tests prove, and what they do not

Command: .\mvnw.cmd "-Dtest=RegressionGateTest,ExperimentComparatorTest,EventVerifierTest,FaultInjectorTest,FaultInjectionExperimentTest" test. Result: 21 run, 0 failures, 0 errors, 0 skipped, BUILD SUCCESS. This covers only the five classes named. It does not prove that every project test passes, and it does not exercise PostgreSQL or Kafka.

Unit tests, no Docker

TEST CLASS	COVERS (FROM THE CLASS NAME, CONFIRM IN SOURCE)	STATUS
EventVerifierTest	Missing, duplicate and ordering detection on in-memory event lists.	Tested
FaultInjectorTest	DROP, DUPLICATE and REORDER behaviour, and repeatability for a given seed.	Tested
FaultInjectionExperimentTest	Injected faults flow through the verifier and are detected. This is the check that the detector actually detects.	Tested
ExperimentComparatorTest	Deltas, and the IMPROVED, REGRESSED, MIXED and UNCHANGED classification.	Tested
RegressionGateTest	Threshold evaluation and the pass or fail decision.	Tested

Integration tests, Docker required

SCENARIO	NEEDS	STATUS
Outbox claim, publish and history against PostgreSQL	PostgreSQL 16 from docker-compose.yml, schema from docker/init.sql	Unverified
Publish and observe round trip	Kafka 3.7 and topic lyapunov.events	Unverified
Full pipeline: database to Kafka to verifier	Both services. Not recently validated, so do not claim it.	Unverified
Two concurrent publishers claiming rows	PostgreSQL. Needed before ordering and SKIP LOCKED work in roadmap Stage 3.	Proposed

Proposed tests for the summary CLI

CASE	EXPECTED BEHAVIOUR	STATUS
Valid reports, several files	Counts totals, reliable and unreliable, and sums per fault type.	Proposed
Malformed JSON	File named in the output, other files still processed. Strict mode exits non-zero.	Proposed
Missing required field	Reported as malformed, never treated as zero.	Proposed
Empty directory	Clear message and a defined exit code.	Proposed
Unsupported fault type	Reported as malformed.	Proposed

Proposed structure

Keep unit tests as the default Maven run. Put Docker-dependent tests behind a separate Maven profile so a missing Docker engine cannot make the normal build fail. Run both in CI, with service containers there instead of a new tool.

Security and operations

Nothing on this board is claimed as implemented unless the Today column says so. Defaults below are development settings, not production-ready.

AREA	TODAY	RECOMMENDATION	STATUS
Externalised configuration	ReliabilityVerifier holds local Kafka and JDBC settings in code. Treated as development settings.	Read bootstrap servers, JDBC URL, topic, timeouts and thresholds from environment variables or a properties file, with the current values as defaults.	Gap
Credential management	Docker Compose uses the development user postgres with password postgres.	Keep that for local development only. Production reads credentials from the environment or a secret store, never from source or reports. Add .env to .gitignore.	Gap
Structured logging	Not confirmed.	One JSON line per lifecycle event with event ID, aggregate, status, worker and experiment seed, so a failure can be traced end to end.	Proposed
Health checks	Not confirmed.	A liveness check for the publisher and a readiness check that tests PostgreSQL and Kafka connectivity.	Proposed
Metrics	None. Reports are the only output.	Counters for claimed, published and failed events. Publish latency. Outbox backlog age. Per-experiment missing, duplicate and ordering counts.	Proposed
Timeouts	The observer has a poll timeout.	Explicit timeouts on JDBC, Kafka producer and consumer calls, all configurable, and recorded in reports when they trigger.	Proposed
Retry policies	Retry counter exists in the schema. Policy not confirmed.	Exponential backoff with jitter and a cap. Safe only because duplicates are tolerated and measured.	Proposed
Dead-letter handling	None.	After the retry cap, move the event to a dead state and keep its history. Start with a status on outbox_events before adding a table.	Proposed
Container health checks	Not confirmed in docker-compose.yml.	Add healthchecks for PostgreSQL (pg_isready) and Kafka so dependent steps wait for readiness. This removes a class of flaky start-up failures.	Proposed
CI testing	Not confirmed. Tests are run by hand.	Build and run unit tests on every push. Run integration tests in a second job. Run an experiment and the --fail-on-regression gate against a stored baseline.	Proposed

Current implementation versus proposed improvements

Current column is what the project brief says exists. Proposed column is not built. Latest confirmed commit: a8cc176 feat: enforce reliability regression gates.

CAPABILITY	CURRENT	PROPOSED	STATUS
Outbox persistence	outbox_events with PENDING, CLAIMED, PUBLISHED, FAILED. event_history audit rows.	Dead state after retry cap. Reclaim after lease expiry.	Exists
Outbox claiming	claimPendingEvents works but does not enforce aggregate head-of-line ordering.	Per-aggregate head-of-line claiming. Concurrency tests with two workers.	Gap
Kafka publication	Topic lyapunov.events on a local broker.	Retry with backoff. Producer timeouts. Aggregate ID as partition key.	Exists
Observation	KafkaEventObserver polls until the expected count or a timeout.	Report timeout as its own cause. Track offsets.	Exists
Verification	Missing, duplicate and ordering checks.	Break missing events down by cause.	Tested
Fault injection	Seeded DROP, DUPLICATE and REORDER.	Process crash, Kafka outage, PostgreSQL outage and poison-event faults.	Tested
Experiments and reports	ExperimentRunner writes indented JSON. executedAt is a String.	Typed timestamp. Schema version field. Report validation.	Exists
Replay	ExperimentReplay reruns from saved seed, fault and count.	Store environment details in the report so differences can be explained.	Exists
Comparison	Deltas and four outcomes.	Comparison written to a file for CI.	Tested
Regression gate	Strict zero-increase thresholds through --fail-on-regression.	Thresholds from a config file. Distinct exit codes.	Tested
Experiment summary	Not built.	Summary CLI with per-file validation and strict mode.	Proposed
Integration validation	Docker-dependent tests unverified. Engine unavailable.	Restore Docker, run the full pipeline once, then wire into CI.	Unverified
Configuration and secrets	Local Kafka and JDBC settings in code. Development credentials.	Externalised configuration and a secret source.	Gap
Observability	Reports only.	Structured logs, health checks, metrics.	Proposed
CI	Not confirmed.	Unit job, integration job, regression-gate job.	Proposed

Roadmap

Proposed work in eight stages. Nothing here is complete. Stages 1 and 2 can run in parallel. Stage 3 needs Stage 2, because ordering and concurrency fixes cannot be trusted without a working database-to-Kafka run.

Stage 1. Experiment summary

Check the package structure and report schema in source, then build the summary CLI with per-file validation and strict mode.
Done when: all nine test cases pass and strict mode exits non-zero on the four truncated reports.

Stage 2. Validate database to Kafka

Restore the Docker engine. Run outbox, publish, observe and verify once, end to end.
Done when: one full run passes and is recorded with date and commit.

Stage 3. Ordering and concurrency

Head-of-line claiming per aggregate. Aggregate ID as partition key. Two-worker tests.
Done when: ordering violations stay at zero with concurrent workers.

Stage 4. Retries and recovery

Backoff, retry cap, dead state, lease reclaim, and an idempotent sample consumer.
Done when: a crash-recovery experiment shows duplicates measured and no missing events.

Stage 5. CI regression checks

Unit, integration and regression-gate jobs against a stored baseline.
Done when: a pull request that regresses a metric fails its build.

Stage 6. Observability

Structured logs, health checks and metrics.
Done when: one event can be traced from creation to verification through logs alone.

Stage 7. Performance and load

Larger event counts. Throughput, publish latency and recovery time.
Done when: baseline numbers are recorded with their environment.

Stage 8. Deployment and docs

Runbook, configuration reference, and architecture documentation from this design.
Done when: a new developer runs the pipeline from the docs alone.

Risks, limitations and open questions

What could make this design wrong, and what to check in the source before building on it.

Risks and limitations

RISK	WHY IT MATTERS	MITIGATION
Design written without reading the source	Class names, behaviour and exact column names come from the brief and a summary of docker/init.sql. Details may differ.	Read the source and init.sql before implementing anything on these boards.
Ordering is not enforced	The verifier can detect ordering violations, but the publisher can cause them.	Roadmap Stage 3.
No end-to-end validation	The PostgreSQL to Kafka path has not run recently, so the headline reliability claim has no recent evidence.	Roadmap Stage 2. Do not claim it before then.
At-least-once crash window	A crash between a Kafka send and the PUBLISHED update produces a duplicate by design.	Measure it. Document it. Provide an idempotent consumer.
Truncated reports	Four old files are malformed. Averages that include or silently skip them are wrong.	Keep them. Flag them. Strict mode fails on them.
Stringly typed timestamp	executedAt as a String invites parse and sort bugs.	Typed timestamp and a schema version in the report.
Synthetic faults only	Tests prove the verifier catches injected list faults, not real outages or crashes.	Add crash and outage faults in later stages.
Replay is partial	The seed fixes the fault, not timing or infrastructure. A replay can still differ.	Record environment details. Compare counts, not timings.
Development credentials in config	Local defaults can leak into shared or deployed setups.	Externalise configuration. Keep real secrets out of Git.

Open engineering questions

QUESTION	WHY IT MATTERS
Where does FaultInjector act: on an in-process event list, or on the Kafka path?	Decides whether experiments test the real pipeline or only the verifier.
Where does ExpectedEventRepository persist expected events?	Decides whether the verifier depends on PostgreSQL and how replay restores expectations.
Is event_history.event ID a declared foreign key?	Affects cleanup, deletion order and audit guarantees.
What happens to CLAIMED rows today if a worker dies?	Lease fields exist. Without reclaim, events can stall silently.
What partition key does the publisher use?	Per-aggregate ordering in Kafka depends on it.
Which consumer group and offset policy does the observer use?	A reused group can skip or re-read events and distort a verdict.
May a threshold ever allow an increase in missing events?	Defines whether tolerances exist for anything except performance metrics.